50. Tunnels: GRE/VXLAN Overlay Awareness

50. Tunnels: GRE/VXLAN Overlay Awareness

Phase: Phase 9 — VPN, tunnels, and overlays
Estimated time: 1–2 days
Slug: `50-gre-vxlan`

Learning objectives

- Explain overlays vs underlays
- Know VXLAN for multi-tenant/datacenter
- See Linux vxlan devices
- Relate to cloud SDN

Prerequisites

Complete prior lessons. Prefer a Linux VM for `ip`/`ss`/`nft` examples; macOS works for many `curl`/`dig` drills. Type every command yourself and keep a lab notes file.

Why this matters

Overlays everywhere: cloud networks, Kubernetes, DC fabrics. GRE/VXLAN awareness explains extra headers, MTU, and 'which IP do ACLs see?'.

Core concepts

What you must be able to explain:

- Explain overlays vs underlays
- Know VXLAN for multi-tenant/datacenter
- See Linux vxlan devices
- Relate to cloud SDN

Mental model for `50-gre-vxlan` (exercise kind `vxlan_quiz`):

- Prefer **measurable** checks (exit codes, ICMP replies, DNS rcode, HTTP status, socket state) over vibes.
- Separate **reachability** (can packets flow?) from **correctness** (right service, cert, route, identity).
- Ask: *which layer's tool falsifies my current hypothesis?* Then run that tool.
- Record the exact command that proved the root cause — that becomes your playbook snippet.

Worked commands

Run these on a lab host. Linux VM preferred for `ip`/`ss`/`tcpdump`/`nft`; macOS is fine for many `curl`/`dig`/`ping` drills. Your addresses and timings will differ — match the *shape* of the output and the conclusion you draw.

Example 1

```
ip -br link; ip route | head
# After VPN up, compare default route and MTU
```

Sample output:

```
wg0 UP ...
default via 10.10.0.1 dev wg0
mtu 1420
```

Interpretation: Tunnels add outer headers → lower effective MTU. Blackhole PMTUD is a classic VPN failure mode.

Example 2

```
ping -c 2 -M do -s 1472 1.1.1.1 2>&1 | head -5 || ping -c 2 -D -s 1472 1.1.1.1
2>&1 | head -5
```

Sample output:

```
Frag needed / Message too long / 100% loss
```

Interpretation: Large DF probes failing after VPN connect points to MTU/MSS issues — lower MSS/MTU or fix ICMP PTB.

Example 3

```
ssh -N -L 8080:127.0.0.1:80 user@bastion  # example local forward (lab)
echo 'local forward uses -L; remote -R; dynamic -D'
```

Sample output:

```
(ssh holds open; curl localhost:8080 hits remote :80)
```

Interpretation: SSH forwards are tactical tunnels. Know `-L` vs `-R` direction before debugging 'connection refused' on the wrong host.

Mini-lab

1. Re-type every worked command for **Tunnels: GRE/VXLAN Overlay Awareness** and paste outputs into lab notes.
2. Introduce one controlled failure in the lab (wrong DNS, firewall drop, bad route) and recover using only this lesson's tools.
3. Write a 5-line playbook: symptom → first command → expected healthy output.
4. Complete the exercise stubs; then explain each function's return value without looking.

Failure modes

Symptom	Likely cause	Next check
Ignoring underlay MTU	See common mistakes	Re-run lesson commands
Debugging only overlay IPs	See common mistakes	Re-run lesson commands
No mapping of VNI/ tenants	See common mistakes	Re-run lesson commands
Timeout with no RST	Silent drop / blackhole	tcpdump + compare path

Study tip for Tunnels: GRE/VXLAN Overlay Awareness

Before each new command, write a one-line hypothesis. If the result falsifies it, update the hypothesis instead of stacking random tools. Note equivalents across OS (`ip` ↔ `ifconfig`/ `route`, `ss` ↔ `netstat`). Re-run two commands from this lesson tomorrow from memory and explain the sample outputs out loud.

Evidence habit

When pages are down, prefer evidence (`ip route get`, resolver answer, packet capture, listener list) over intuition. Document **command → output → conclusion** in a lab notebook so the same path is reusable on-call. Change one variable at a time; write a one-line hypothesis before each new tool.

Common mistakes

- Ignoring underlay MTU
- Debugging only overlay IPs
- No mapping of VNI/tenants

Exercises

Work in `exercises/50-gre-vxlan/`. Read that folder's `README.md` before coding.

```
cd exercises/50-gre-vxlan
chmod +x test.sh exercise.sh 2>/dev/null || true
./test.sh
```

What you will build

Edit `exercise.sh` and implement `vxlan_is_an`, `vxlan_over`, `gre_is_a`. Each function is a tiny retrieval drill for **Tunnels: GRE/VXLAN Overlay Awareness** — return the exact tokens/ flags/labels from the lesson. Keep the comment above each function describing when you would use that fact on-call. Run `./test.sh` until all 3 assertions pass.

Study loop

- **A (Drill):** make tests pass
- **B (Apply):** run 2–3 real commands from this lesson in your lab and note outputs
- **C (Challenge):** rewrite answers from memory next day

Do not open `solutions/50-gre-vxlan/` until after an honest attempt.

Self-check

Answer without scrolling up:

1. Overlay vs underlay?
2. VXLAN common use?
3. Why MTU again?

Further reading

- https://www.rfc-editor.org/rfc/rfc7348